

Buffer Pool Manager

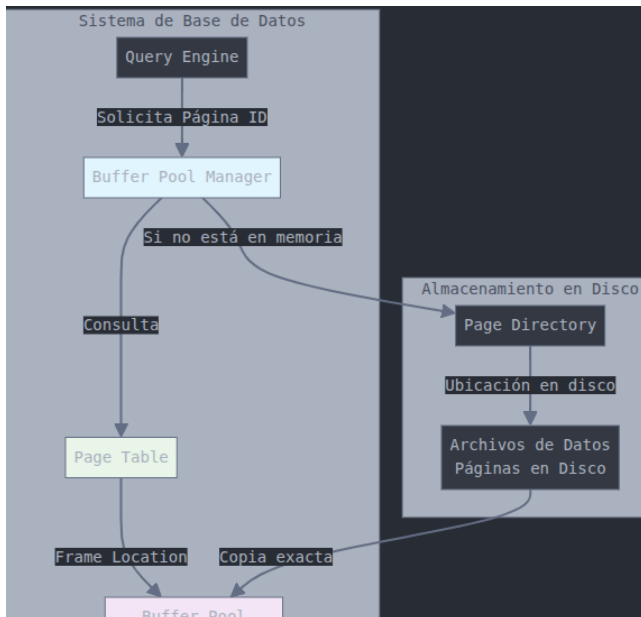
Gestion de Memoria

Wilson Ramos Pacco

Universidad Nacional de San Agustín de Arequipa

20 de junio de 2025

Arquitectura General





PageTable

Es una estructura EN MEMORIA (volátil) que implementa el mapeo crucial entre PageID → FrameID.

```
// Hash map principal: PageID
PageTableEntry
std::unordered_map<int, PageTableEntry> table;

struct PageTableEntry {
    int frame_id;
    int pin_count;
    bool dirty_bit;
    bool valid_bit;
    time_point last_access_time;
};

\\GESTION DE PINING
bool pinPage(int page_id);
bool unpinPage(int page_id);
\\GESTION DE ESTADOS
bool markDirty(int page_id);
bool clearDirty(int page_id);
```

TEST: GESTIÓN DE PIN/UNPIN

```
=====
Page Table: Página 10 → Frame 0
Page Table: Página 20 → Frame 1
Page Table: Página 30 → Frame 2
```

1. Estado Inicial

```
-----
Page Table: [30-F2, 20-F1, 10-F0]
```

2. Operaciones de PIN

```
-----
Haciendo pin de página 10...
★ Pin página 10 (count: 1)
Haciendo pin de página 10 otra vez...
★ Pin página 10 (count: 2)
Haciendo pin de página 20...
★ Pin página 20 (count: 1)
Page Table: [30-F2, 20-F1(P1), 10-F0(P2)]
✓ Página 10 tiene pin_count = 2
✓ Página 20 tiene pin_count = 1
```

3. Operaciones de UNPIN

```
-----
Haciendo unpin de página 10...
↑ Unpin página 10 (count: 1)
✓ Página 10 ahora tiene pin_count = 1
Haciendo unpin de página 10 otra vez...
↑ Unpin página 10 (count: 0)
✓ Página 10 ahora tiene pin_count = 0
```

4. Verificación de Estados de Evicción

```
-----
Páginas evictables: 30 10
✓ 2 páginas son evictables
Page Table: [30-F2, 20-F1(P1), 10-F0]
```

PageTable

```
-----  
Marcando página 100 como dirty...  
📄 Página 100 marcada como DIRTY  
✅ Página 100 ahora está DIRTY  
Marcando página 200 como dirty...  
📄 Página 200 marcada como DIRTY  
Page Table: [200→F1(D), 100→F0(D)]
```

3. Verificar Estadísticas de Dirty Pages

```
-----
```

```
✅ 2 páginas están dirty
```

4. Limpiar Dirty Bits

```
-----
```

```
Limpiando dirty bit de página 100...  
🌟 Página 100 marcada como CLEAN  
✅ Página 100 ahora está CLEAN
```

TEST: GESTIÓN DE TIMING PARA LRU

```
=====
```

```
📁 Page Table: Página 1 → Frame 0  
📁 Page Table: Página 2 → Frame 1  
📁 Page Table: Página 3 → Frame 2
```

1. Accesos con Delays para Simular LRU

```
-----
```

```
Accediendo página 1...  
Accediendo página 2...  
Accediendo página 3...  
Re-accediendo página 1...
```

2. Verificar Orden LRU

```
-----
```

```
Página LRU (candidata para evicción): 2  
Página 1: 0ms desde último acceso  
Página 2: 200ms desde último acceso  
Página 3: 100ms desde último acceso
```

```
=== PAGE TABLE (MEMORIA) ===  
Total páginas: 3  
Páginas pinned: 0  
Páginas dirty: 0  
Páginas evictables: 3
```

PageTable

```
-----  
Marcando página 100 como dirty...  
📄 Página 100 marcada como DIRTY  
✅ Página 100 ahora está DIRTY  
Marcando página 200 como dirty...  
📄 Página 200 marcada como DIRTY  
Page Table: [200→F1(D), 100→F0(D)]
```

3. Verificar Estadísticas de Dirty Pages

```
-----
```

```
✅ 2 páginas están dirty
```

4. Limpiar Dirty Bits

```
-----
```

```
Limpiando dirty bit de página 100...  
🌟 Página 100 marcada como CLEAN  
✅ Página 100 ahora está CLEAN
```

TEST: GESTIÓN DE TIMING PARA LRU

```
=====
```

```
📁 Page Table: Página 1 → Frame 0  
📁 Page Table: Página 2 → Frame 1  
📁 Page Table: Página 3 → Frame 2
```

1. Accesos con Delays para Simular LRU

```
-----
```

```
Accediendo página 1...  
Accediendo página 2...  
Accediendo página 3...  
Re-accediendo página 1...
```

2. Verificar Orden LRU

```
-----
```

```
Página LRU (candidata para evicción): 2  
Página 1: 0ms desde último acceso  
Página 2: 200ms desde último acceso  
Página 3: 100ms desde último acceso
```

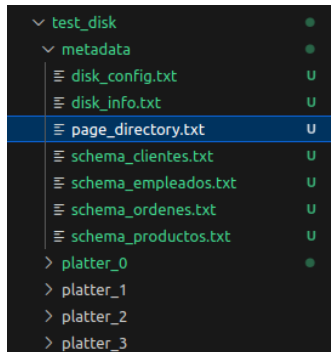
```
=== PAGE TABLE (MEMORIA) ===  
Total páginas: 3  
Páginas pinned: 0  
Páginas dirty: 0  
Páginas evictables: 3
```

PageDirectory

El PageDirectory es el componente PERSISTENTE del sistema de gestión de páginas

- PageDirectory (DISCO): Mapeo PageID → ubicación física en archivos
- PageTable (MEMORIA): Mapeo PageID → FrameID en buffer pool

```
class PageDirectory {
private:
    std::unordered_map<int, PageLocation> directory
    ;
    std::string directory_file;
    bool is_dirty;
    struct PageLocation {
        std::string file_id;        // Identificador del
            archivo f sico
        size_t offset;              // Offset dentro del
            archivo
        size_t size;                // Tama o de la
            p gina
    };
};
```



```
# Page Directory - Mapeo PageID - (FileID, Offset, Size)
# Generado automáticamente por DiskManager
total pages=18
next page_id=20
19=PD_S0_TO_SEC10|0|4096
13=PD_S0_TO_SEC12|0|4096
12=PD_S0_TO_SEC11|0|4096
11=PD_S0_TO_SEC10|0|4096
10=PD_S0_TO_SEC9|0|4096
17=PD_S0_TO_SEC16|0|4096
4=PD_S0_TO_SEC3|0|4096
16=PD_S0_TO_SEC15|0|4096
```

Frame Title

```
Registro insertado en tabla 'viviendas' (ID: 32, Tiempo: 19.9123 ms)
Registro insertado en tabla 'viviendas' (ID: 33, Tiempo: 1.36911 ms)
Registro insertado en tabla 'viviendas' (ID: 34, Tiempo: 12.1198 ms)
■ Page Directory: Registrada página 2 → P0_S0_T0_SEC1
Registro insertado en tabla 'viviendas' (ID: 35, Tiempo: 13.154 ms)
Registro insertado en tabla 'viviendas' (ID: 36, Tiempo: 9.07524 ms)
Registro insertado en tabla 'viviendas' (ID: 37, Tiempo: 14.1403 ms)
Registro insertado en tabla 'viviendas' (ID: 38, Tiempo: 15.1692 ms)
Registro insertado en tabla 'viviendas' (ID: 39, Tiempo: 13.8858 ms)
Registro insertado en tabla 'viviendas' (ID: 40, Tiempo: 10.2963 ms)
Registro insertado en tabla 'viviendas' (ID: 41, Tiempo: 11.6865 ms)
Registro insertado en tabla 'viviendas' (ID: 42, Tiempo: 11.8514 ms)
Registro insertado en tabla 'viviendas' (ID: 43, Tiempo: 7.36734 ms)
Registro insertado en tabla 'viviendas' (ID: 44, Tiempo: 11.6476 ms)
Registro insertado en tabla 'viviendas' (ID: 45, Tiempo: 12.2837 ms)
Registro insertado en tabla 'viviendas' (ID: 46, Tiempo: 10.5155 ms)
Registro insertado en tabla 'viviendas' (ID: 47, Tiempo: 17.6157 ms)
Registro insertado en tabla 'viviendas' (ID: 48, Tiempo: 13.4303 ms)
Registro insertado en tabla 'viviendas' (ID: 49, Tiempo: 7.2774 ms)
Registro insertado en tabla 'viviendas' (ID: 50, Tiempo: 7.60365 ms)
Registro insertado en tabla 'viviendas' (ID: 51, Tiempo: 12.7311 ms)
Registro insertado en tabla 'viviendas' (ID: 52, Tiempo: 3.44028 ms)
Registro insertado en tabla 'viviendas' (ID: 53, Tiempo: 9.76176 ms)
```



```
=== PAGE DIRECTORY (GESTIONADO POR DISK MANAGER) ===
```

```
=== PAGE DIRECTORY (DISCO) ===
```

```
Archivo: mi_disco_sgbd/metadata/page_directory.txt
```

```
Páginas registradas: 17
```

```
Estado: Modificado
```

```
Páginas:
```

```
Página 17 → P0_S0_T0_SEC16 (offset: 0, size: 4096)
```

```
Página 16 → P0_S0_T0_SEC15 (offset: 0, size: 4096)
```

```
Página 15 → P0_S0_T0_SEC14 (offset: 0, size: 4096)
```

```
Página 14 → P0_S0_T0_SEC13 (offset: 0, size: 4096)
```

```
Página 1 → P0_S0_T0_SEC0 (offset: 0, size: 4096)
```

```
Página 2 → P0_S0_T0_SEC1 (offset: 0, size: 4096)
```

```
Página 3 → P0_S0_T0_SEC2 (offset: 0, size: 4096)
```

```
Página 4 → P0_S0_T0_SEC3 (offset: 0, size: 4096)
```

```
Página 5 → P0_S0_T0_SEC4 (offset: 0, size: 4096)
```

```
Página 6 → P0_S0_T0_SEC5 (offset: 0, size: 4096)
```

```
Página 7 → P0_S0_T0_SEC6 (offset: 0, size: 4096)
```

```
Página 8 → P0_S0_T0_SEC7 (offset: 0, size: 4096)
```

```
Página 9 → P0_S0_T0_SEC8 (offset: 0, size: 4096)
```

```
Página 10 → P0_S0_T0_SEC9 (offset: 0, size: 4096)
```

```
Página 11 → P0_S0_T0_SEC10 (offset: 0, size: 4096)
```

BUFFER POOL MANAGER

Problema: Acceder al disco es 1000x más lento que acceder a memoria RAM.

Solución: BufferPoolManager mantiene un “cache inteligente” que:

- Guarda páginas frecuentes en memoria (rápido)
- Solo va al disco cuando es absolutamente necesario (lento)
- Evicta páginas menos usadas cuando se llena la memoria

Funciones Principales:

```
auto block = buffer_manager->requestPage(5, PageOperation::READ);
```

¿Qué hace?

1. Busca en memoria primero (PageTable)
2. Si está: HIT → Retorna inmediatamente
3. Si no está: MISS → Busca en disco y carga
4. Actualiza estructuras (LRU, estados)

BUFFER POOL MANAGER

```
std::shared_ptr<Block> requestPage(int page_id, PageOperation operation) {
    std::cout << "\n Solicitando página " << page_id
                << " para " << (operation == PageOperation::READ ? "LECTURA" : "ESCRITURA")
                << std::endl;

    // 1. Verificar si está en Page Table (memoria)
    PageTableEntry entry;
    if (page_table->findPage(page_id, entry)) {
        std::cout << "Página " << page_id << " encontrada en memoria (Frame "
                    << entry.frame_id << ")" << std::endl;

        // Pin la página
        page_table->pinPage(page_id);

        // Actualizar LRU
        lru_replacer->recordAccess(entry.frame_id);

        // Marcar como dirty si es operación de escritura
        if (operation == PageOperation::WRITE) {
            page_table->markDirty(page_id);
            buffer_pool[entry.frame_id].is_dirty = true;
        }

        if (operation == PageOperation::READ) {
            read_operations++;
        } else {
            write_operations++;
        }
    }
}
```